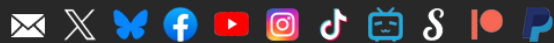


## Inigo Quilez :: articles :: raymarching distance fields - 2008



Raymarching SDFs (Signed Distance Fields) is slowly getting popular, because it's a simple, elegant and powerful way to represent 3D objects and even render 3D scenes. SDFs are a type of implicit surface representation, and so its origins can be tracked back very far in the history of computer graphics. While not SDFs per-se, the first paper that's been brought to my attention showing boolean operations in implicit surfaces by combining them through the `min()` and `max()` operators dates back to 1972, by A.Ricci and later by [B.Wyvill](#) and [G.Wyvill](#) in 1989. The first mention of raymarching SDFs seems to be also from that time - the paper by [Sandin, Hart and Kauffman](#), which used it for rendering 3D fractals, although they never coined the term "SDF". Seven years later in 1995 [C.Hart](#) himself documented the raymarching technique once again (but unfortunately miscalled it "Sphere Tracing" - usually we trace rays, cylinders or cones, but definitely not spheres!).

My story with SDFs starts around 2001. Like many other demosceners of that time, I experimented a lot with raymarching Signed Fields (without the "D") to render fractals and metaballs as part of our demos. However it was only after I read [Alex Evan's work](#) from 2006 and [Keenan Crane's work](#) from 2005 that I got excited about exploring the consequences of constraining a Signed Field to be an Euclidean Distance. And so, in 2007 I dived in, and explored, and then produced the first results that we'd recognizable as modern SDF, including the four images at the bottom of this page that demonstrated non-trivial but art directed content, and demonstrated techniques for efficient soft shadows, smooth blending and domain repetition.



A raymarched procedural SDF, step by step modeling, shading and lighting

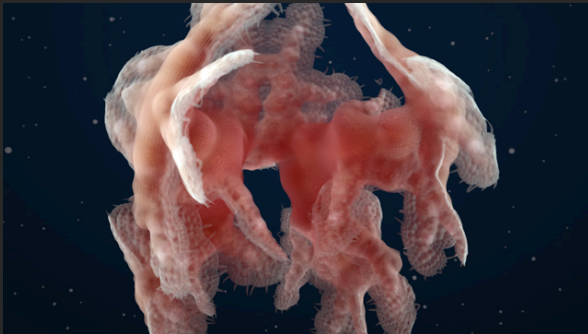
Now, although raymarching SDFs can definitely **render regular polygonal meshes**, as a lover of compression and procedural content myself, my focus became to implement the SDFs directly as list of primitives rather than as volumetric data, which is where many folks' first instinct seem to go, despite the unresolvable precision issues, probably because their mindset is that of rendering in video games. However, primitive based SDF offer the infinite sharpness and precision that is crucial both for CAD and professional graphic design. Unlike volume based approaches of constant access time, rendering unlimited number of primitives and their CSG (boolean) operations that define the SDF requires some acceleration structures and more complex techniques than volumes. But coincidentally, back in 2007, the timing for such a procedural/math and compute based approach was great - First, GPUs were evolving their compute/ALU capabilities faster than their memory bandwidth, a trend that has only continued in the last 20 years, so pure mathematical SDFs started to be competitive against a backed representations. Second, I personal had an interest in doodling around and finding the required maths to express different shapes and effects. Third, I was also very much interested in improving my artistic skills, so procedural content was a great fit. And fourth, as a demoscener, I was naturally captivated by procedural content generation and its potential for reduced memory footprint (RAM and disk). As it turns, pretty rendering, procedural content and small memory footprint was also in high demand at Pixar, I learnt a couple of years later, but that's a story for another day.

And so, for the next 5 years, the demoscene and the Shadertoy communities we pushed the SDF raymarching technique further, creating more and better looking content. With each new demo and Shadertoy piece, we proved SDFs did not have to be limited to just funny fractals or metaballs or toy cities of ever repeating buildings, but content could be created that was art directed, and included sophisticated modeling, proper shading with filtering, lighting and animation. As the volume of works grew, a case started to emerge for SDFs as a legit professional representation for

3D graphics, at least in certain applications. And about 5 years later academia noticed this, and started working on the area as well. Another 5 years or so later the industry itself also joined in, and we started seeing the emergence of professional modeling and design tools based on SDFs.

Anyways, to end this already long post, since 1998 I've been documenting some of the discoveries/inventions I did around SDFs, or just tricks or simple observations that could be interesting to others, both in this very website and my [Youtube channel](#). So here are some pointers if you want to start learning the basics of SDFs:

- [Articles on Raymarching](#) on this site
- [My video tutorials and streams about raymarching SDFs](#) on Youtube
- The lecture called "Rendering Worlds with Two Triangles" that I have 2008 on the topic
- [All my Shadertoy examples](#), some of which I list below:



**Sea Creature, 2022**

This was a quick one-nighter. It uses a classic KIFS recursive fractal schema exactly like the Angels painting below, but here I used smooth-minimum and smooth-abs to blend all spheres together into a single organic shape. The rest is color tweaking. The shader runs slow because I'm doing volumetric rendering so I get some sweet transparencies( as usual I'm okay with doing something 4x slower if that makes it look 10% better!)

Source code:

<https://www.shadertoy.com/view/csB3zy>



**Selfie Girl, 2020**

This was my second human SDF and my first attempt at facial animation. When I finished it I was as proud of it as I was critical of the many flaws it had. But of course I think it's still a great example of using SDFs for organic modeling. I think the whole primitive count for the whole painting is 32, so that gives you a sense on how little data is needed to represent 3D scenes and how efficiently it can be stored and processed.

Source code:

<https://www.shadertoy.com/view/WsSBzh>

**Tutorial:** <https://www.youtube.com/watch?v=8-5LwHRhjk>



**Sphere Gears, 2019**

This was an improv session, based on some other Shadertoy user's idea, which I started as an optimization exercise. In particular, thanks to the symmetry of the object you can evaluate only 4 gear pieces instead of the 18 that make the whole object. Similarly, within each gear, only a single tooth/dent is evaluated instead of 12, making the whole SDF pretty cheap to evaluate.

Source code:

<https://www.shadertoy.com/view/tt2XzG>

**Tutorial (part 1):**

<https://www.youtube.com/watch?v=sl9x19EnKnG>

**Tutorial (part 2):**

<https://www.youtube.com/watch?v=bdlCU2uvOdU>



**Planet Fall, 2018**

Planet Fall was an improv/jam session, although it turned out pretty well. It's a level recursion of voronoi distributed cylinders on a sphere. The



**Happy Jumping, 2019**

Happy Jumping was a one-day effort to animate a character, which I had never done before really. I designed a super simple character for it and the animation was rough - in fact, it gets out of model very often. But it was a fine first attempt. The lighting is my usual 3 light rig (key, fill, bounce) and some fake subsurface.

Source code:

<https://www.shadertoy.com/view/3lsSzf>

**Tutorial:** <https://www.youtube.com/watch?v=Cfe5UQ-1L9Q>

<https://www.youtube.com/watch?v=Cfe5UQ-1L9Q>



**Surfer Boy, 2018**

This was my first attempt at creating a human face completely procedurally/mathematically. It's made mostly of ellipsoids, cones and quadratic

lighting is cheated as usual and the occlusion mostly painted by hand. But the color palette makes it look so pretty.

Source code:

<https://www.shadertoy.com/view/ltBWB>



**Greek Temple, 2017**

This image was born as a live coding improv session for the students of UPenn, although I spend a couple of days working after the live coding was done. The temple is made of basic domain repetition of 6 or 7 boxes and cylinders. The most interesting bit is probably the lack of global illumination - instead, the rich bounce lighting in the temple is achieved by coloring the relevant areas by hand positioned colors. Also, for the sake of an interesting composition, the light direction that illuminates the ocean and terrain is different from the direction of the light that illuminates the temple.

Source code:

<https://www.shadertoy.com/view/ldScDh>

**Tutorial:** <https://www.youtube.com/watch?v=-pdSjBPH3zM>

bezier curves. It was mostly modeled to camera - it doesn't look that good from other angles. In order to keep it realtime I did a pretty simple lighting setup (no ambient occlusion, no GI, just painted colors here and there).

Source code:

<https://www.shadertoy.com/view/ldd3DX>

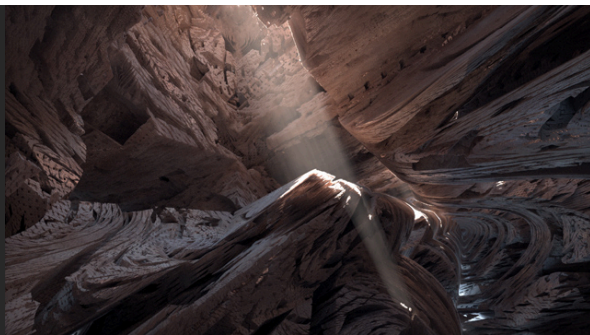


**Ladybug, 2017**

I made this one after another live coding session for the students of the University of Washington. I made a mushroom for them, and then the two weeks that followed, in small chunks of night time, I slowly completed the drawings until I got here. The modeling is weak and I only pushed it enough to sell the image at medium resolution. It could get a lot more love, but I had to stop somewhere before I'd take too many nights. Lighting is mostly faked (no ambient occlusion, no GI) and very much painted by hand to look good (especially everything that has to do with bounce lighting and subsurface looking materials).

Source code:

<https://www.shadertoy.com/view/4tByz3>

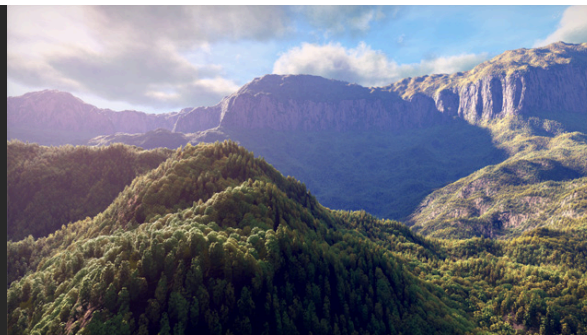


### Fractal Cave, 2016

The was a **menger sponge fractal** with some variations and a heavy domain distortion. The image was path-traced by raymarching the signed distance field.

Source code:

<https://www.shadertoy.com/view/Xtt3Wn>



### Rainforest - 2016

This was an attempt to raymarch a signed distance field of a tree-populated terrain. Trees are mere spheres with some basic noise displacement distributed on a voronoi pattern. The most interesting part is the heavy use of analytical derivatives of noise to compute fast normals and slopes, which helped a lot speed up the raymarching process of both terrain (the bigger the slope, the smaller the step size - you can read on the Lipschitz constant).

**Tutorial:** <https://www.youtube.com/watch?v=BFId4EBO2RE>

Source code:

<https://www.shadertoy.com/view/4ttSWf>



### Elephants - 2016

Since the Snail worked so well, I tried to make elephants too, with the same exact techniques. This proved to be more difficult, and the results are only mediocre. In particular, the modeling was difficult to make for me by just typing



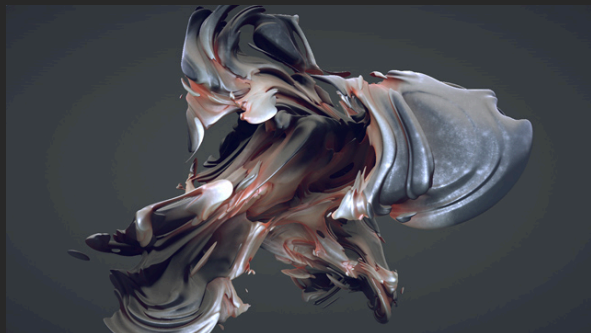
### Snail, 2015

The Snail was one of my all time favorite images, because it looks good and tells a little story. It was mostly a modeling exercise, by using simple **distance functions** blended together **smoothly**. The shell was interesting to

formulas, so I left it unfinished and only added detail that would be seen from this camera angle. The trees are ellipsoids with some domain distortion, and the terrain a distance field to a plane with displacement.

Source code:

<https://www.shadertoy.com/view/4dKGWm>



**Sculpture III, 2015**

Just like Sculpture II below, this was an (domain) deformed sphere through [here](#). The distortion was made with four octaves of sine waves.

Source code:

<https://www.shadertoy.com/view/XtjSDK>

make, and the translucent feeling of the snail itself took me a while to get right.

Source code:

<https://www.shadertoy.com/view/ld3Gz2>

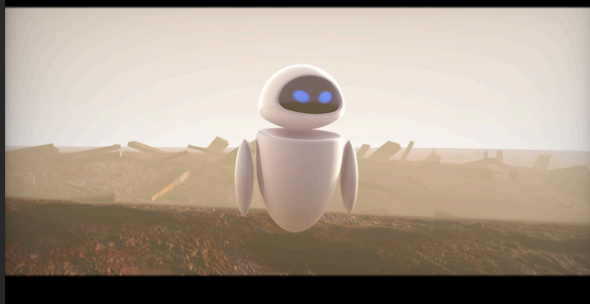


**Arlo - 2015**

This was mostly a procedural modeling exercise with signed distance field. By using simple [distance functions](#) blended together [smoothly](#), I was able to replicate the main character of Pixar's movie The Good Dinosaur. I didn't have time to pose it in a more interesting action, so he's looking forward and stiff. The most interesting part in this drawing was the use of ray differentials to do [proper texture filtering](#) in a raymarching context.

Source code:

<https://www.shadertoy.com/view/4dtGWM>



### Eve arrives, 2015

Eve was my first attempt at modeling an actual character. And I chose her because she is simple - she's made of basic ellipsoids and sphere that are **smooth blended** together. The background detail is done with domain repetition of one box.

Source code:

<https://www.shadertoy.com/view/lIsXRX>

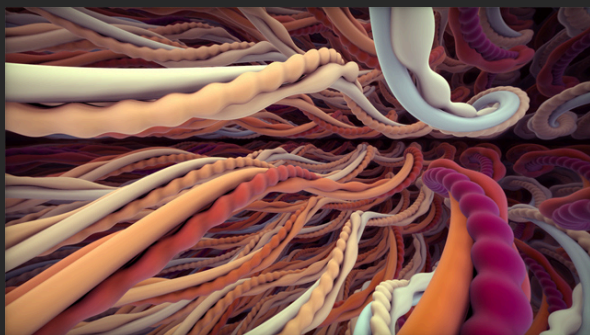


### Mushroom - 2015

For this drawing I tried to create rocks by a technique that I saw the user TDM use for one of **his shaders**: start with a sphere and cut bigger spheres of it to create a semi sharp rock. It worked more or less fine, I got some interesting rocks.

Source code:

<https://www.shadertoy.com/view/4tBXR1>



### Antialias, sort of - 2014

This was an attempt to antialiasing the screen-space edges of the raymarched distance fields. By using ray differentials, one can estimate how much of the pixel footprint (pixel frustum) does approximately intersect the geometry, given that at each marching step we have the distance to the closest surface. That was used in this image to compute partial coverage per near-intersection, and then they were all composited front to back to give the correct smooth image without edge pixelization.



### Sculpture II - 2014

This was another quick experiment on domain distortion through **here**. In this case, a sphere got distorted by a sine wave of a sine wave of a sine wave of a sine wave... of a sine wave. The result: a beautiful shape.

Source code:

<https://www.shadertoy.com/view/4ssSRX>

Source code:

<https://www.shadertoy.com/view/llXGR4>.



**Canyon - 2014**

A simple terrain in this case. Some bits are floating in the air since it was defined as a true 3D field, not as a height-map based distance field. Adding white snow on a red/yellow terrain always works and it's a cheap way to get good results without doing much effort on actual shading/texturing/surfacing.

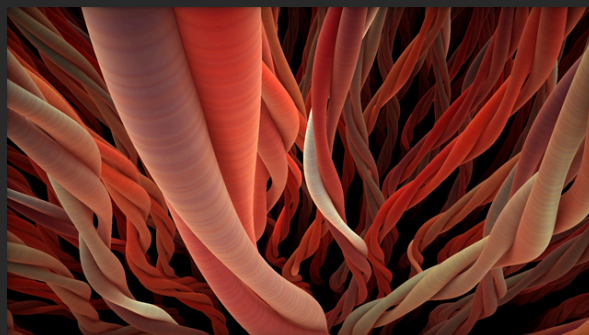
Source code:

<https://www.shadertoy.com/view/MdBGzG>



**Fish Swimming - 2014**

The interesting aspect of this signed distance field image is the way the texturing was done. Since there are no meshes or vertices for the modeling of the fish, there is no place to store UV coordinates that can deform with the model



**Worms - 2014**

This one was improvised over a night, and it featured some domain repetition with only three cylinders modeled, and domain distortion to make them curve along the vertical axis.

Source code:

<https://www.shadertoy.com/view/XsjXR1>



**Dolphin - 2014**

This was my attempt at doing water splashes. That was done by doing some displacement of the water plane with an exponential shape based on the proximity of the dolphin to the water. Indeed, the signed distance field

itself. But I was able to make the animation function of the fish invertible such that each point on the surface would be able to know where it came from in the rest pose of the fish and do the noise and pattern lookups there. That way the textures didn't swim on the surface of the fish as they did in the Insect drawing below. The fish did swim actually, with some simple sine waves and noise.

Source code:

<https://www.shadertoy.com/view/l dj3Dm>



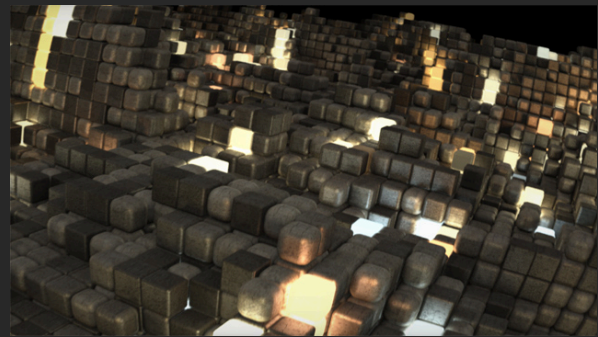
**Woods - 2013**

In this case a simple camera rotation was enough to hide the domain repetition of the trees. The interesting part is that I had to fake the lighting to the point of bending the light itself along the depth of the drawing: the direction of the sun light is different in the foreground and in the background (interpolated between those two poses). This allowed me to get the patch of light in the red mushroom by the camera and the nice side lighting in the tree in the background. Modeling wise, the trees are cylinders with an exponentially decaying radius along its axis, all smoothly blended together with the **smooth minimum formula**, and some noise on top as a displacement.

representation can be used beyond finding marching and shadow or occlusion computation - in this case helps the water know about the dolphin. The dolphin itself was made with a few cylinders and some other smoothly blended primitives.

Source:

<https://www.shadertoy.com/view/4sS3zG>



**Rounded Voxels - 2013**

This was an exercise to integrate together fast ray-casting through a regular grid and raymarching inside the grid cells. Basically, the ray marches quickly through a volumetric regular grid, which can be done efficiently with a few additions and integer mask operations, and when found a grid cell to be not empty, a raymarcher takes over to find the intersection of the signed distance field defining the rounded boxes. If no intersection happens, the control is returned to the grid marcher.

Source code:

<https://www.shadertoy.com/view/4djGWR>

Source code:

<https://www.shadertoy.com/view/XsfGD4>



**Volcanic - 2013**

This was a serious attempt at getting some decent outdoors lighting without resorting to global illumination. I wrote an article about it [here](#). Modeling was based mostly on value noise, and there's also some smoke added as a second raymarching pass.

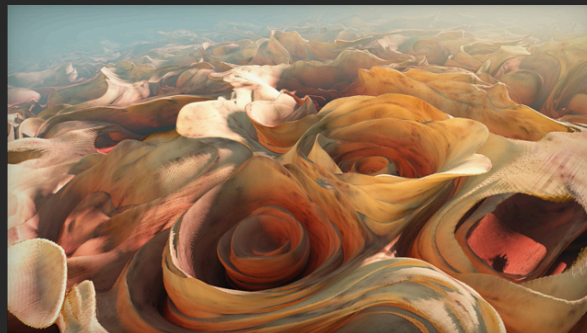
Source code:

<https://www.shadertoy.com/view/XsX3RB>



**Angels - 2013**

Angels used some basic domain repetition to create many flying creatures despite I only had one. The differences in the animations between the different creatures was made by taking the cell id to offset the animation formula (which was based on cosine waves). The creatures themselves were made with some simple fractal



**Wavy - 2013**

This was an attempt to pretty colors. It used some **domain deformation** through wrapping like the one I described [here](#), but in 3D and with only one level of indirection (the rendering is slow enough as it is).

Source code:

<https://www.shadertoy.com/view/ldX3zS>



**Insect - 2013**

This shader was one of the first to use the **smooth minimum formula** to blend shapes together (legs and body). It also performs some analytic inverse kinematic to position the legs automatically in the terrain without simulation or integration.

recursion of ellipsoids.

Source code:

<https://www.shadertoy.com/view/lssGRM>



**Piano - 2013**

This was an exercise in combining true lighting with painted lighting. By "painted" I mean that it wasn't computed through shadow casting or pathtracing, but that its color was injected artificially in specific areas and objects in the scene to convey ("fake") expensive lighting effects. For example, some of the light in the bottom part of the wall was added artificially, and the window from which the light comes in doesn't exist, but was shaped by hand with the formula of a square. On the other hand, this drawing performed true reflections.

Source code:

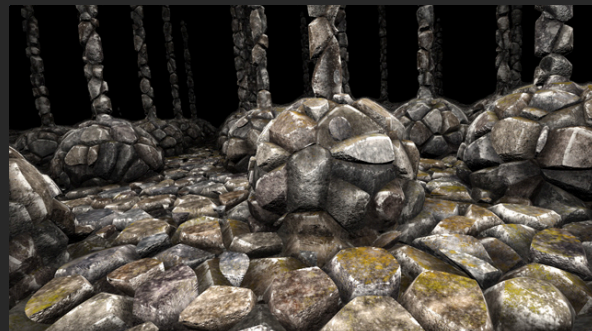
<https://www.shadertoy.com/view/ldl3zN>



**Fruxis - 2012**

Source code:

<https://www.shadertoy.com/view/Mss3zM>

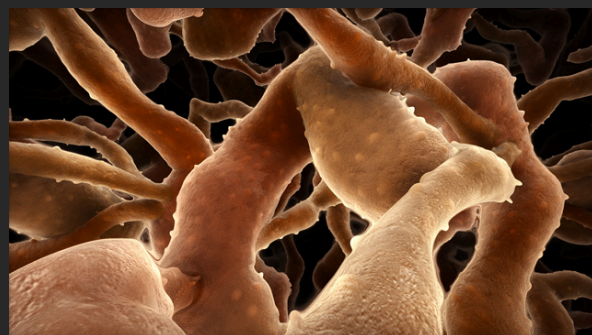


**Rocks - 2013**

This drawing was based on a voronoi pattern again, like Leizex (see below), and was mostly an exercise on shading simple rocks. However it's interesting to see how now this could be done in real-time in the GPU while Leizex (2008) wasn't real time at all!

Source code:

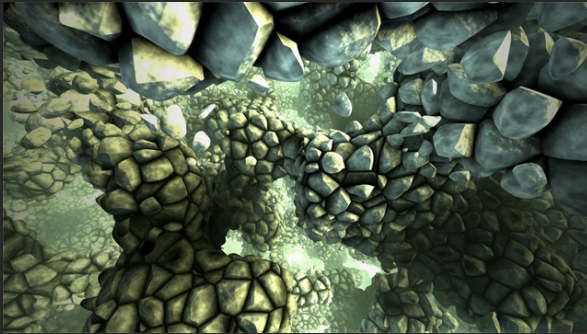
<https://www.shadertoy.com/view/MsXGzM>



**Cell - 2013**

I made this one for the Trsac Demo party. It was my raymarched procedural distance field that was implemented in the GPU (GLSL). It was also my first to use a pathtracer (using [this algorithm](#)). You can find a modified version of it that does **not** use pathtracing here:

<https://www.shadertoy.com/view/ldl3zl>



**Leizex - 2008**

This was a one day production. I wanted to check how it is to directly raymarch a three dimensional voronoi pattern. It did work, although it was very slow then, but it is realtime this days of course. The lighting is fake again - there aren't any light sources, neither there is ambient occlusion computations or anything. Colors are procedurally assigned to points in space.

Source code here:

<https://www.shadertoy.com/view/XtycD1>

This was my first procedural signed distance field exercise in Shadertoy (WebGL). The interesting bit was my attempt to fake the subsurface scattering of light by raymarching inside the volume of the geometry to measure thickness.

Source code here:

<https://www.shadertoy.com/view/Xdl3R4>



**Bridge - 2009**

This is a sketch really, but it never got completed, due to the lack of artistic inspiration at the time. The interesting part is probably the technique used for creating the grass, which is based on voronoi-driven domain tiling.

**Organix - 2008**

Organix won the 1st position at the Function Demoparty in Hungary. It's a raymarched SDF, but it run on the CPU in around 40 seconds at 1280x720 resolution. Needless to say this is realtime these days in the GPU. The lighting is fake in this one, in that shadows and ambient occlusion are procedurally painted on the geometry, not computer through raycasting or similar techniques.

Source Code:

<https://www.shadertoy.com/view/ldByDh>

**Slisesix - 2008**

This image was the first image I generated by raymarching SDFs (Signed Distance Functions). It contained smooth blending of geometric primitives, domain repetition, ambient occlusion and soft shadows. It won the 1st prize of 4 Kilobytes Procedural Graphics of Euskal Party 2008 in Spain.

Source code here:

<https://www.shadertoy.com/view/lsf3zr>

**inigo quilez** - learning computer graphics since 1994